模型能力声明与路由策略技术方案

1. 文档信息

- 文档状态：待实施
- 方案版本：V1.0
- 编写日期：2026-09-01
- 适用范围：Mia 模型配置、Agent 配置、Chat/Agent Runtime 与 Run Trace
- 首期范围：文本对话与图片理解模型路由

2. 背景

Mia 当前由 Agent 绑定一个 Chat 模型。上传图片时，只有绑定支持图片输入的多模态模型才能正常处理；如果主模型只支持文本，系统缺少统一的能力校验、自动视觉模型选择和可观测路由记录。

ChatGPT、豆包等成熟产品通常将用户看到的助手与底层模型解耦：模型声明自身能力，产品根据输入类型和业务策略选择实际执行模型。对于 Mia，不应要求每个 Agent 都绑定专用图片模型，也不应让 LLM 自由决定首次模型选择。图片是否存在、模型是否支持图片均可由代码确定性判断。

本方案建立以下分层：

```
模型配置负责声明能力
Agent 配置负责业务偏好
Runtime 负责确定性路由
Provider Client 负责供应商协议转换
Run Trace 负责记录路由依据与实际模型
```

3. 目标与非目标

3.1 目标

1. 模型配置可以声明文本、图片、工具调用、思考等能力。
2. Agent 可以设置图片处理策略，但不需要默认绑定第二个模型。
3. 纯文本始终优先使用 Agent 主模型。
4. 图片输入优先使用支持视觉的主模型，否则按 Agent 和系统策略选择视觉模型。
5. 模型选择过程确定、可解释、可审计。
6. Run Trace 可以显示模型路由原因、候选模型和实际执行模型。
7. Provider 差异只在模型模块内部处理，不污染 Chat 和 Agent Runtime。
8. 为音频、图片生成、推理模型和成本路由预留扩展边界。

3.2 P0 非目标

- 不建设通用智能模型网关。
- 不按成本和负载在多个同能力模型间动态竞价。
- 不让模型自主选择第一次调用的底层模型。
- 不实现音频、视频和图片生成路由。
- 不实现复杂历史图片语义引用。
- 不自动信任通过模型名称猜测出的能力。

4. 设计原则

4.1 能力与用途分离

`modelType=CHAT` 表示模型用途；`VISION_INPUT` 表示能力。视觉语言模型仍属于 Chat 模型，不新增 `VISION` 模型类型。

4.2 确定性路由优先

输入附件类型和模型能力由代码判断。路由发生在模型调用前，不额外调用 LLM 做分类，避免增加延迟、费用和不确定性。

4.3 主模型优先

主模型已支持全部输入能力时继续使用主模型，避免不必要切换、上下文差异和第二次模型调用。

4.4 配置错误显式失败

Agent 禁止图片、视觉模型停用或系统未配置默认视觉模型时返回明确业务错误，不允许主模型忽略图片后继续猜测。

4.5 路由只选择模型，不编排业务

模型路由不负责 OCR、RAG、Tool Loop 或 Workflow。OCR 等垂直能力应作为工具或独立 Workflow 实现。

5. 目标架构

```
ChatController / ExternalAccessController
                  |
                  v
            ChatService
       session/history/SSE mapping
                  |
                  v
           AgentRuntime
       analyze input capabilities
                  |
                  v
      ModelProvidersService.resolveChatModel
       capability + policy + defaults
                  |
                  v
       ModelProvidersGateway
                  |
                  v
         ProviderClient Adapter
     OpenAI / Anthropic / Gemini / Ollama
```

模块边界：

- `agents`：保存 Agent 主模型与视觉策略，返回稳定运行时 DTO。
- `model_providers`：拥有模型能力、默认模型、路由规则和供应商 Client。
- `agent_runtime`：分析本次输入需要的能力，调用模型路由并记录 Step。
- `chat`：拥有附件、历史消息、会话和 SSE 映射，不查询模型能力表。
- `agent_runs`：保存路由决策和实际模型调用信息。

6. 数据模型

6.1 模型能力枚举

```
enum ModelCapabilityType {
  TEXT_CHAT
  VISION_INPUT
  TOOL_CALLING
  REASONING
  STRUCTURED_OUTPUT
  AUDIO_INPUT
  AUDIO_OUTPUT
  IMAGE_GENERATION
  EMBEDDING
  RERANK
}
```

P0 只启用 `TEXT_CHAT` 和 `VISION_INPUT`，其他枚举为后续阶段预留。

6.2 模型能力表

```
model ModelCapability {
  id            String              @id @default(uuid()) @db.Uuid
  modelConfigId String              @map("model_config_id") @db.Uuid
  capability    ModelCapabilityType
  enabled       Boolean             @default(true)
  options       Json?               @db.JsonB
  verified      Boolean             @default(false)
  verifiedAt    DateTime?            @map("verified_at") @db.Timestamptz(6)
  createdAt     DateTime            @default(now()) @map("created_at") @db.Timestamptz(6)
  updatedAt     DateTime            @updatedAt @map("updated_at") @db.Timestamptz(6)
  modelConfig   ModelConfig         @relation(fields: [modelConfigId], references: [id], onDelete: Cascade)

  @@unique([modelConfigId, capability])
  @@index([capability, enabled])
  @@map("model_capabilities")
}
```

视觉能力的 `options` 示例：

```
{
  "maxImages": 10,
  "maxImageBytes": 10485760,
  "supportedMimeTypes": ["image/jpeg", "image/png", "image/webp"],
```

```
  "imageTransport": "data_url"
}
```

`verified=false` 表示管理员手动声明但尚未经过供应商接口验证。Mia 不应仅凭模型名称自动认定能力。

6.3 系统默认模型

在现有 `DefaultModelSetting` 增加：

```
visionModelConfigId String? @map("vision_model_config_id") @db.Uuid
visionModelConfig   ModelConfig? @relation("DefaultVisionModel", fields: [visionModelConfigId], references: [id],
onDelete: SetNull)
```

默认视觉模型必须同时满足：

- `modelType=CHAT`；
- 模型与供应商均启用且未删除；
- 能力包含启用的 `VISION_INPUT`。

6.4 Agent 图片策略

```
enum AgentVisionMode {
  AUTO
  PRIMARY_ONLY
  DEDICATED
  DISABLED
}

model Agent {
  visionMode          AgentVisionMode @default(AUTO) @map("vision_mode")
  visionModelConfigId String?         @map("vision_model_config_id") @db.Uuid
}
```

模式	行为
`AUTO`	主模型支持视觉时使用主模型，否则使用系统默认视觉模型
`PRIMARY_ONLY`	只允许主模型处理图片，不支持时明确失败
`DEDICATED`	图片请求固定使用 Agent 指定的视觉模型
`DISABLED`	Agent 不允许图片输入

P0 前端只需展示“自动选择、指定视觉模型、禁止图片”。`PRIMARY_ONLY` 可放入高级配置或 P1 再开放。

7. 跨模块 DTO

7.1 Agent 运行时配置

```
type AgentRuntimeConfig = {
  agentId: string;
  modelConfigId: string;
  vision: {
    mode: "AUTO" | "PRIMARY_ONLY" | "DEDICATED" | "DISABLED";
    visionModelConfigId: string | null;
  };
};
```

7.2 路由请求

```
type ResolveChatModelRequest = {
  primaryModelConfigId: string;
  requiredCapabilities: ModelCapabilityType[];
  policy: {
    visionMode: AgentVisionMode;
    visionModelConfigId: string | null;
  };
  requestId: string;
};
```

7.3 路由结果

```ts
type ModelRouteDecision = {
  selectedModelConfigId: string;
  selectedRole: "PRIMARY" | "SYSTEM_VISION" | "AGENT_VISION";
  requiredCapabilities: ModelCapabilityType[];
  reason:
    | "PRIMARY_SUPPORTS_ALL"
    | "AGENT_VISION_SELECTED"
    | "SYSTEM_VISION_FALLBACK";
  fallbackUsed: boolean;
};
```

DTO 不返回 Prisma Model、供应商密钥或 Provider Client。

8. 路由规则

8.1 输入能力识别

P0 使用确定性识别：

```
只有文本                    -> TEXT_CHAT
包含 image/* 附件           -> TEXT_CHAT + VISION_INPUT
```

附件类型必须根据服务端验证后的 MIME 判断，不信任文件名后缀或客户端传入类型。

8.2 路由算法

```ts
async function resolveChatModel(input, agent, primary, defaults) {
  if (!input.hasImages) return primary;

  if (agent.vision.mode === "DISABLED") {
    throw new AgentVisionDisabledError();
  }

  if (agent.vision.mode === "DEDICATED") {
    return requireVisionModel(agent.vision.visionModelConfigId);
  }

  if (agent.vision.mode === "PRIMARY_ONLY") {
    return requireVisionCapability(primary);
  }

  if (primary.supports("VISION_INPUT")) return primary;
  return requireVisionModel(defaults.visionModelConfigId);
}
```

决策顺序：

```
消息包含图片?
├─ 否: 主对话模型
└─ 是
   ├─ Agent 禁止图片: 明确失败
   ├─ Agent 指定视觉模型: 指定模型
   ├─ 主模型支持视觉: 主模型
   ├─ 已配置系统默认视觉模型: 默认视觉模型
   └─ 无可用模型: 明确失败
```

8.3 Fallback 约束

- 只允许在首 Token 前切换候选模型。
- 首 Token 后失败不得自动换模型续写。
- `PRIMARY_ONLY` 和 `DISABLED` 不得被系统默认视觉模型绕过。
- 内容安全拦截、图片格式错误和图片超限不得自动降级。
- 所有 Fallback 必须写入 Run Trace。

9. 多模态消息标准化

业务模块使用供应商无关结构：

```
type ChatContentPart =
  | { type: "text"; text: string }
  | {
      type: "image";
      mimeType: string;
      storageId: string;
      dataUrl?: string;
    };
```

Provider Client 分别转换为供应商协议：

- OpenAI-compatible：`image_url`；
- Anthropic：`image/source/base64`；
- Gemini：`inlineData`；
- Ollama：对应模型支持的 `images` 格式。

Chat 和 Agent Runtime 禁止直接构造 `image_url`，否则会与 OpenAI-compatible 协议耦合。

10. 上下文与附件策略

自动路由到视觉模型后仍携带相同文本历史。P0 图片上下文采用以下规则：

1. 当前轮图片必须携带。
2. 历史文本正常按 Agent `historyTurns` 裁剪。
3. 历史图片默认不无限重复携带。
4. 最近一轮图片最多在后续两轮中继续附带。
5. Trace 和普通日志不保存图片 Base64，只保存附件 ID、MIME、大小和哈希摘要。

P1 可增加显式图片引用和按语义决定是否重新附带历史图片。

11. API 设计

11.1 模型能力管理

```
GET  /api/v1/model-configs/:id/capabilities
PUT  /api/v1/model-configs/:id/capabilities
POST /api/v1/model-configs/:id/capabilities/verify
```

更新示例：

```
{
  "capabilities": [
    { "type": "TEXT_CHAT", "enabled": true },
    {
      "type": "VISION_INPUT",
      "enabled": true,
      "options": {
        "maxImages": 10,
        "maxImageBytes": 10485760,
        "supportedMimeTypes": ["image/jpeg", "image/png", "image/webp"]
      }
    }
  ]
}
```

11.2 默认视觉模型

复用现有默认模型接口，扩展字段：

```
{
  "chatModelConfigId": "...",
  "embeddingModelConfigId": "...",
  "visionModelConfigId": "..."
}
```

11.3 Agent 配置

创建和更新 Agent 增加：

```
{
  "visionMode": "AUTO",
```

```
  "visionModelConfigId": null
}
```

Service 必须验证 `DEDICATED` 模式下模型存在、启用且包含 `VISION_INPUT`。

12. Run Trace 与日志

每次模型调用前新增路由步骤：

```
#1 ROUTER · model_route
#2 MODEL  · chat_stream
```

`model_route` 输入：

```
{
  "primaryModelConfigId": "...",
  "requiredCapabilities": ["TEXT_CHAT", "VISION_INPUT"],
  "visionMode": "AUTO",
  "attachmentCount": 1
}
```

输出：

```
{
  "selectedModelConfigId": "...",
  "selectedRole": "SYSTEM_VISION",
  "reason": "SYSTEM_VISION_FALLBACK",
  "fallbackUsed": true
}
```

`chat_stream` Step 记录实际 `modelConfigId`、模型名、供应商、耗时、首 Token 和用量。不得记录 API Key、Authorization、图片 Base64 或完整敏感附件地址。

13. 前端设计

13.1 模型配置

新增“模型能力”区域：

```
模型能力
☑ 文本对话
☑ 图片理解
☑ 工具调用
☑ 结构化输出
☑ 可选思考
```

图片理解高级配置包括支持格式、单张上限和单次最大数量，并标识“已验证/未验证”。

13.2 系统默认模型

增加“默认视觉模型”。下拉框只展示启用、供应商可用且声明 `VISION_INPUT` 的 Chat 模型。

13.3 Agent 配置

在主模型下增加：

```
图片处理：自动选择 / 指定视觉模型 / 禁止图片
```

选择“指定视觉模型”后显示视觉模型下拉框。界面同时展示路由摘要：

```
纯文本：qwen3.7-max
图片：doubao-vision-pro
```

14. 错误模型

错误码	用户提示
AGENT_VISION_DISABLED	当前 Agent 未启用图片处理
PRIMARY_MODEL_NO_VISION	当前主模型不支持图片输入
DEFAULT_VISION_MODEL_NOT_CONFIGURED	系统尚未配置默认视觉模型
VISION_MODEL_DISABLED	配置的视觉模型已停用
UNSUPPORTED_IMAGE_TYPE	不支持该图片格式
IMAGE_TOO_LARGE	图片超过模型允许的大小
TOO_MANY_IMAGES	图片数量超过模型限制
MODEL_CAPABILITY_MISMATCH	模型声明能力与供应商实际能力不一致

禁止统一转换为“模型调用失败”。

15. 安全、容量与成本

1. 上传时检查真实 MIME、图片数量、单张大小和总大小。
2. 必要时清理 EXIF，避免泄露拍摄位置等元数据。
3. 图片必须通过 Workspace 和会话归属校验后才能读取。
4. 不在日志、Trace、Prompt 预览中记录 Base64。
5. 删除会话或附件时按现有存储生命周期清理文件。
6. 视觉模型独立记录 Token、耗时与估算成本。
7. 图片 Data URL 不进入通用缓存键或错误堆栈。
8. 供应商请求取消和超时继续复用现有 Gateway 机制。

16. 迁移与兼容

16.1 数据迁移

1. 为所有启用 Chat 模型补充 `TEXT_CHAT` 能力。
2. 现有模型不自动补充 `VISION_INPUT`，由管理员确认。
3. 所有 Agent 默认迁移为 `visionMode=AUTO`。
4. 默认视觉模型初始为空，不影响纯文本对话。
5. 旧 Agent 收到图片且主模型没有视觉能力时返回明确配置提示。

16.2 API 兼容

- 新字段均提供服务端默认值。
- 旧前端不提交视觉字段时仍可创建和更新 Agent。
- 现有文本 Chat、Workflow LLM 节点和公开 Widget 的纯文本行为保持不变。
- Workflow LLM 节点的多模态路由不纳入 P0，后续复用同一 `resolveChatModel` 能力。

17. 实施计划

P0：图片自动路由闭环

1. 新增模型能力枚举、表和迁移。
2. 增加模型能力管理 API 与前端配置。
3. 增加默认视觉模型配置。
4. 增加 Agent 图片策略和运行时 DTO。
5. 实现 `resolveChatModel` 确定性路由。
6. 标准化多模态消息并适配 OpenAI-compatible Client。
7. 增加 `model_route` Run Step。
8. 增加图片格式、大小、数量和权限校验。
9. 覆盖主模型直用、系统回退、Agent 指定和禁止图片测试。

P1：能力完善

- 工具调用与结构化输出能力声明。
- 将现有思考模式逐步统一到模型能力体系。
- Provider 能力探测与验证状态。
- 历史图片引用与重新附带策略。
- Workflow LLM 节点复用模型路由。

P2：高级路由

- 推理模型、音频和图片生成能力。
- 按模型健康状态、成本与延迟选择候选模型。
- 管理员可配置的路由策略模板。

18. 测试与验收

18.1 Service 测试

- 纯文本始终选择主模型。
- 主模型支持视觉时不发生切换。
- `AUTO` 在主模型不支持视觉时选择系统视觉模型。
- `DEDICATED` 只选择 Agent 指定模型。
- `PRIMARY_ONLY` 不允许系统回退。
- `DISABLED` 拒绝图片输入。
- 模型或供应商停用时返回明确错误。
- 模型能力与图片格式、数量、大小校验正确。

18.2 Provider Client 测试

- OpenAI-compatible 图片消息转换正确。
- Anthropic/Gemini 后续适配不得影响统一 DTO。
- 请求取消、超时和错误分类保持有效。
- 日志和错误不包含图片 Base64。

18.3 端到端验收

1. 文本 Agent 行为和改造前一致。
2. 多模态主模型可以直接处理图片。
3. 文本主模型可以自动路由至默认视觉模型。
4. Agent 可以指定专用视觉模型或禁止图片。
5. Run Trace 可以看到 `model_route` 和实际 `chat_stream` 模型。
6. 无可用视觉模型时用户获得可操作的配置提示。

19. P0 最终调用链

```
Chat 接收消息
  -> 校验附件类型、大小、数量和归属
  -> AgentsService.getAgentRuntimeConfig
  -> AgentRuntime 分析 requiredCapabilities
  -> ModelProvidersService.resolveChatModel
  -> AgentRunsService 记录 model_route Step
  -> 构造统一多模态 ChatMessage
  -> ModelProvidersGateway
  -> ProviderClient 转换供应商协议
  -> 流式返回并记录实际模型、首 Token、用量和路由原因
```

该方案先解决图片理解的最小自动路由闭环，不提前引入微服务或复杂模型网关；后续新增音频、图片生成和推理模型时继续扩展统一能力声明与路由 DTO。